Mini Project: Airport Code Lookup Service with a Binary Search Tree

Karishma Premnath Mithila Krishna Shashikala
EECE 7205

Task 1: Load the Data and Build the Tree

(a) Load the data

Splitting on every comma breaks line 633, `"Harstad/Narvik Airport, Evenes"`, and line 654, `"Tromsø Airport,"`. `splitLine` reads character by character and remembers whether it is inside quotes. Counting from 0, the IATA code is `f[4]`.

```
SPLIT-LINE(line): main steps
1.  fields ← empty list;  field ← "";  inQuotes ← FALSE
2.  for i ← 0 to line.size − 1 do
3.      c ← line[i]
4.      if c = '"' then
5.          if inQuotes and line[i + 1] = '"' then
6.              field ← field + '"';  i ← i + 1
7.          else inQuotes ← not inQuotes
8.      else if c = ',' and not inQuotes then
9.          append field to fields;  field ← ""
10.     else field ← field + c
11. append field to fields
12. return fields
```

```
LOAD(fileName, records): main steps
1.  open fileName;  if it fails then return FALSE
2.  skipped ← 0;  totalLines ← 0;  firstLine ← TRUE
3.  while getline(in, line) do
4.      if firstLine then
5.          firstLine ← FALSE
6.          if line starts with "airport_id" then continue
7.      totalLines ← totalLines + 1
8.      f ← SPLIT-LINE(line)
9.      if f.size < 9 then continue
10.     if f[4] = "\N" or f[4] = "" then skipped ← skipped + 1;  continue
11.     a ← new Airport
12.     a.name ← f[1];  a.city ← f[2];  a.country ← f[3]
13.     a.iata ← f[4];  a.icao ← f[5]
14.     a.lat ← to_double(f[6]);  a.lon ← to_double(f[7])
15.     append a to records
16. return TRUE, skipped, totalLines
```

`airports.dat` has no header and writes a missing code as `\N`. `airports.csv` has a header row and leaves the field empty. The loader takes either one.

Result: 6,072 airports loaded, 1,626 lines skipped.

(a) Check the loader against the posted facts

```
COUNT-COUNTRIES(records): main steps
1.  seen ← empty list
2.  for i ← 0 to records.size − 1 do
3.      found ← FALSE
4.      for j ← 0 to seen.size − 1 do
5.          if seen[j] = records[i].country then found ← TRUE; break
6.      if not found then append records[i].country to seen
7.  return seen.size
```

```
COUNT-IN-COUNTRY(records, country): main steps
1.  total ← 0
2.  for i ← 0 to records.size − 1 do
3.      if records[i].country = country then total ← total + 1
4.  return total
```

Result: 7,698 lines, 6,072 codes, 1,626 skipped, 235 countries, 1,251 in the United States. All five match the handout.

(b) Build the BST in file order

Insert is a loop, not recursion: the sorted build in Task 6 is a chain of 6,072 nodes, height 6,071.

```
INSERT(key, record): main steps
1.  create newNode with key and record
2.  if root = NULL then root ← newNode; return INSERTED
3.  cur ← root
4.  loop
5.      if key < cur.key then
6.          if cur.left = NULL then cur.left ← newNode; return INSERTED
7.          cur ← cur.left
8.      else if key > cur.key then
9.          if cur.right = NULL then cur.right ← newNode; return INSERTED
10.         cur ← cur.right
11.     else return DUPLICATE
```

```
BUILD-FROM-ARRAY(records, n): main steps
1.  for i ← 0 to n − 1 do
2.      if INSERT(records[i].iata, records[i]) = DUPLICATE then
3.          report records[i].iata as a duplicate
```

Result: 6,072 keys inserted, 0 duplicates. Root is GKA.

(c) Height and average depth

Height counts edges: an empty tree is -1, a single node 0. COLLECT walks the tree in level order with a growing list. Neither measurement recurses.

```
COLLECT(nodes, depths): main steps
1.  if root = NULL then return
2.  append root to nodes;  append 0 to depths
3.  for i ← 0 to nodes.size − 1 do
4.      node ← nodes[i];  d ← depths[i]
5.      if node.left  ≠ NULL then append node.left  to nodes; append d + 1
    to depths
6.      if node.right ≠ NULL then append node.right to nodes; append d + 1
    to depths
```

```
HEIGHT(): main steps
1.  if root = NULL then return −1
2.  COLLECT(nodes, depths)
3.  maxDepth ← 0
4.  for i ← 0 to depths.size − 1 do
5.      if depths[i] > maxDepth then maxDepth ← depths[i]
6.  return maxDepth
```

```
AVERAGE-DEPTH(): main steps
1.  if n = 0 then return 0
2.  COLLECT(nodes, depths)
3.  total ← 0
4.  for i ← 0 to depths.size − 1 do
5.      total ← total + depths[i]
6.  return total / n
```

Result: Height 177, average depth 18.3.

Output

```
$ g++ -std=c++17 -O2 TASK1.cpp -o TASK1
$ ./TASK1 data/airports.dat
```

```
===== Task 1(a): load the data =====
Loaded 6072 airports (1626 lines skipped: no IATA code).

Loader checked against the facts in the handout:
  lines in total         7698      expected 7698
  3-letter IATA codes    6072      expected 6072
  skipped (no code)      1626      expected 1626
  countries covered      235       expected 235
  in the United States   1251      expected 1251

splitLine on the two names that contain a comma:
  line 633:
    name = [Harstad/Narvik Airport, Evenes]
    iata = [EVE]   icao = [ENEV]
  line 654:
    name = [Tromsø Airport,]
    iata = [TOS]   icao = [ENTC]

One stored record:
  BOS  General Edward Lawrence Logan International Airport, Boston, United States

===== Task 1(b): build the BST in file order =====
Keys inserted: 6072
Duplicates:    0
Root:          GKA
Nodes in tree: 6072

===== Task 1(c): shape of the tree =====
Height:        177
Average depth: 18.3
```

Data: OpenFlights.org, `airports.dat`, Open Database License.

Task 2: Search

(a) find CODE

The path is collected during the descent. Its length is the comparison count.

```
FIND(target, path, record): main steps
1.  path ← empty list
2.  cur ← root
3.  while cur ≠ NULL do
4.      append cur.key to path
5.      if target = cur.key then record ← cur.data; return FOUND
6.      else if target < cur.key then cur ← cur.left
7.      else cur ← cur.right
8.  return NOT FOUND
```

Result: `find BOS` returns the airport in 12 comparisons, the value given in the handout.

(b) Every search case

The leaf and the deep code are found by walking the tree, not hardcoded.

```
A-LEAF-KEY(): main steps
1.  COLLECT(nodes, depths)
2.  for i ← 0 to nodes.size − 1 do
3.      if nodes[i].left = NULL and nodes[i].right = NULL then return nodes[
    i].key
4.  return "(none)"
```

```
DEEPEST-KEY(): main steps
1.  COLLECT(nodes, depths)
2.  best ← 0
3.  for i ← 0 to depths.size − 1 do
4.      if depths[i] > depths[best] then best ← i
5.  return nodes[best].key
```

The smallest key is the leftmost node and the largest is the rightmost.

```
MIN-KEY(): main steps
1.  if root = NULL then return "(empty)"
2.  cur ← root
3.  while cur.left ≠ NULL do cur ← cur.left
4.  return cur.key

MAX-KEY(): main steps
1.  if root = NULL then return "(empty)"
2.  cur ← root
3.  while cur.right ≠ NULL do cur ← cur.right
4.  return cur.key
```

Case	Code	Comparisons	Result
the root	GKA	1	found
a leaf	AAC	5	found
a code deep in the tree	YZR	178	found
a code that does not exist	XYZ	14	not found
smaller than every key	AA	4	not found
larger than every key	ZZZ	37	not found

Result: minimum code AAA, maximum code ZZV.

No three-letter code sits below AAA, so the "smaller than every key" case uses `AA`. Its search ends at AAA with the left child empty.

YZR costs 178 comparisons against 12 for BOS. One comparison per level, which is the $O(h)$ cost of a search.

Output

```
$ g++ -std=c++17 -O2 TASK2.cpp -o TASK2
$ ./TASK2 data/airports.dat
```

```
Loaded 6072 airports. Height 177.

===== Task 2(a): find CODE, with path and comparisons =====

> find BOS   (the example from the handout)
  Path: GKA -> AEY -> EGS -> BUJ -> BJA -> BSK -> BOY -> BNI -> BOH -> BOO ->
        BOU -> BOS
  Comparisons: 12
  BOS  General Edward Lawrence Logan International Airport, Boston, United States
```

```
===== Task 2(b): every search case =====

> find GKA   (the root)
  Path: GKA
  Comparisons: 1
  GKA  Goroka Airport, Goroka, Papua New Guinea

> find AAC   (a leaf)
  Path: GKA -> AEY -> AAE -> AAA -> AAC
  Comparisons: 5
  AAC  El Arish International Airport, El Arish, Egypt

> find YZR   (a code deep in the tree)
  Path: GKA -> MAG -> POM -> WWK -> YAM -> YAY -> YAZ -> YBB -> YBC -> YBG ->
        YBK -> YBL -> YBR -> YCB -> YCD -> YCG -> YCH -> YCL -> YCO -> YCT ->
        YCW -> YCY -> YZS -> YDA -> YDB -> YDF -> YDL -> YDN -> YDQ -> YEG ->
        YEK -> YEN -> YET -> YEU -> YEV -> YFB -> YFC -> YFE -> YFO -> YFR ->
        YFS -> YGK -> YGL -> YGP -> YGQ -> YGR -> YHB -> YHD -> YHI -> YHK ->
        YHM -> YHU -> YHY -> YHZ -> YIB -> YIO -> YJN -> YJT -> YKA -> YKF ->
        YKL -> YKY -> YKZ -> YLD -> YLJ -> YLL -> YLT -> YLW -> YMA -> YMJ ->
        YMM -> YMO -> YMW -> YMX -> YNA -> YND -> YNM -> YOC -> YOD -> YOJ ->
        YOW -> YPA -> YPE -> YPG -> YPL -> YPN -> YPQ -> YPR -> YPY -> YQA ->
        YQB -> YQF -> YQG -> YQH -> YQK -> YQL -> YQM -> YQQ -> YQR -> YQT ->
        YQU -> YQV -> YQW -> YQX -> YQY -> YQZ -> YRB -> YRI -> YRJ -> YRM ->
        YRT -> YSB -> YSC -> YSJ -> YSM -> YSR -> YSU -> YSY -> YTE -> YTH ->
        YTR -> YTS -> YTZ -> YUB -> YUL -> YUT -> YUX -> YUY -> YVC -> YVG ->
        YVM -> YVO -> YVP -> YVQ -> YVR -> YVT -> YVV -> YWA -> YWG -> YWK ->
        YWL -> YWY -> YXC -> YXD -> YXE -> YXH -> YXJ -> YXL -> YXP -> YXR ->
        YXS -> YXT -> YXU -> YXX -> YXY -> YYB -> YYC -> YYD -> YYE -> YYF ->
        YYG -> YYH -> YYJ -> YYL -> YYN -> YYQ -> YYR -> YYT -> YYU -> YYW ->
        YYY -> YYZ -> YZD -> YZE -> YZF -> YZH -> YZP -> YZR
  Comparisons: 178
  YZR  Chris Hadfield Airport, Sarnia, Canada

> find XYZ   (a code that does not exist)
  Path: GKA -> MAG -> POM -> WWK -> YAM -> XFW -> XGN -> XLS -> XRY -> XVS ->
        YAI -> YAA -> XYA -> XYE -> (NULL)
  Comparisons: 14
  not found

> find AA   (smaller than every key)
  Path: GKA -> AEY -> AAE -> AAA -> (NULL)
  Comparisons: 4
  not found

> find ZZZ   (larger than every key)
  Path: GKA -> MAG -> POM -> WWK -> YAM -> YAY -> YAZ -> YBB -> YBC -> YBG ->
        YBK -> YBL -> YBR -> YCB -> YCD -> YCG -> YCH -> YCL -> YCO -> YCT ->
        YCW -> YCY -> YZS -> YZT -> YZU -> YZV -> YZW -> YZX -> ZFA -> ZFM ->
        ZND -> ZQW -> ZSE -> ZVA -> ZWA -> ZZU -> ZZV -> (NULL)
  Comparisons: 37
  not found

Minimum code: AAA
Maximum code: ZZV
```

Data: OpenFlights.org, `airports.dat`, Open Database License.

Task 3: Delete

(a) remove CODE

Delete has three cases at the node it finds. A leaf is unhooked. A node with one child is replaced by that child. A node with two children keeps its position but takes the key and record of its inorder successor, and the successor is then deleted from the right subtree.

```
DELETE(node, key, caseUsed): main steps
1.  if node = NULL then return NULL
2.  if key < node.key then node.left ← DELETE(node.left, key, caseUsed)
3.  else if key > node.key then node.right ← DELETE(node.right, key,
    caseUsed)
4.  else
5.      if node.left = NULL and node.right = NULL then
6.          caseUsed ← "D1 leaf";  free node;  return NULL
7.      if node.left = NULL then
8.          caseUsed ← "D2 only a right child";  child ← node.right
9.          free node;  return child
10.     if node.right = NULL then
11.         caseUsed ← "D2 only a left child";  child ← node.left
12.         free node;  return child
13.     caseUsed ← "D3 two children"
14.     s ← node.right
15.     while s.left ≠ NULL do s ← s.left
16.     node.key ← s.key;  node.data ← s.data
17.     node.right ← DELETE(node.right, s.key, ignored)
18. return node
```

Step 16 copies the record as well as the key. Copying only the key would leave the node holding the successor's code against the old airport's name.

Result: `remove BOS` reports `D2: only a left child, BOR`, which is the line given in the handout.

(b) One code for each case, taken from the tree

```
FIND-CASE(kind): main steps
1.  COLLECT(nodes, depths)
2.  for i ← 0 to nodes.size − 1 do
3.      hasLeft  ← nodes[i].left  ≠ NULL
4.      hasRight ← nodes[i].right ≠ NULL
5.      if kind = LEAF       and not hasLeft and not hasRight then return
    nodes[i].key
6.      if kind = LEFT-ONLY  and hasLeft and not hasRight     then return
    nodes[i].key
7.      if kind = RIGHT-ONLY and not hasLeft and hasRight     then return
    nodes[i].key
8.      if kind = TWO        and hasLeft and hasRight and i > 0 then return
    nodes[i].key
9.  return "(none)"
```

```
LEVEL-ORDER-TOP(levels): main steps
1.  if root = NULL then return
2.  COLLECT(nodes, depths)
3.  for d ← 0 to levels − 1 do
4.      print "level d:"
5.      for i ← 0 to nodes.size − 1 do
6.          if depths[i] = d then print nodes[i].key
```

```
IS-BST(node, lo, hi): main steps
1.  if node = NULL then return TRUE
2.  if lo is set and node.key ≤ lo then return FALSE
3.  if hi is set and node.key ≥ hi then return FALSE
4.  return IS-BST(node.left, lo, node.key) and IS-BST(node.right, node.key,
    hi)
```

Case	Code	Reported
D1 a leaf	AAC	D1: a leaf
D2 only a left child	GJT	D2: only a left child, GJR
D2 only a right child	AAA	D2: only a right child, AAC
D3 two children	AEY	D3: two children, successor AFA
the root	GKA	D3: two children, successor GKE

Each delete runs on a freshly built tree. The cases overlap otherwise: AAC is both the leaf and the right child of AAA, so removing one changes the case the other falls into.

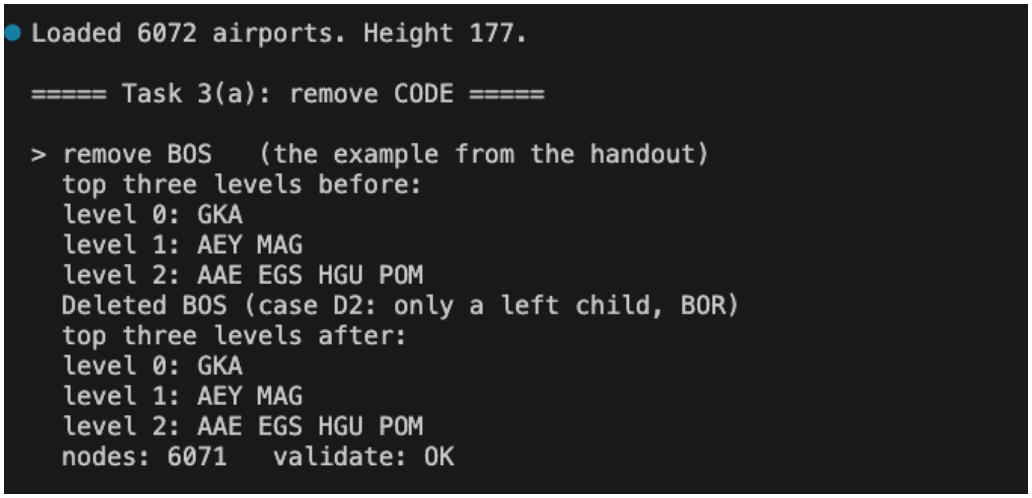
Two deletes change the top levels: removing AEY puts AFA at level 1, and removing the root puts GKE at level 0. The other three sit below level 2 and leave the top unchanged. `validate` is OK after every delete.

(c) A made-up airport

QQQ is inserted with my own name and city, found, removed and confirmed gone. It arrives as a leaf, so its deletion is case D1 and the count returns to 6,072.

Output

```
$ g++ -std=c++17 -O2 TASK3.cpp -o TASK3
$ ./TASK3 data/airports.dat
```



```
===== Task 3(b): one code from the tree for each case =====
  D1 a leaf              : AAC
  D2 only a left child   : GJT
  D2 only a right child  : AAA
  D3 two children        : AEY
  the root               : GKA

> remove AAC   (D1, a leaf)
  top three levels before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  Deleted AAC (case D1: a leaf)
  top three levels after:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  nodes: 6071   validate: OK

> remove GJT   (D2, only a left child)
  top three levels before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  Deleted GJT (case D2: only a left child, GJR)
  top three levels after:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  nodes: 6071   validate: OK

> remove AAA   (D2, only a right child)
  top three levels before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  Deleted AAA (case D2: only a right child, AAC)
  top three levels after:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  nodes: 6071   validate: OK

> remove AEY   (D3, two children)
  top three levels before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  Deleted AEY (case D3: two children, successor AFA)
  top three levels after:
  level 0: GKA
  level 1: AFA MAG
  level 2: AAE EGS HGU POM
  nodes: 6071   validate: OK

> remove GKA   (the root)
  top three levels before:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  Deleted GKA (case D3: two children, successor GKE)
  top three levels after:
  level 0: GKE
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  nodes: 6071   validate: OK
```

```
===== Task 3(c): add QQQ, find it, then remove it =====

> add QQQ
  inserted: yes   nodes: 6073   validate: OK

> find QQQ
  QQQ  Karishma Premnath Airport, Boston, United States
  Comparisons: 22

> remove QQQ
  Deleted QQQ (case D1: a leaf)
  nodes: 6072   validate: OK

> find QQQ again
  not found, as expected
```

Task 4: Traversals and Validation

(a) list, in alphabetical code order

Inorder of a BST comes out in increasing key order. Checking that the list is strictly increasing tests the whole tree, not just the traversal.

```
INORDER(out): main steps
1.  stack ← empty;  cur ← root
2.  while cur ≠ NULL or stack is not empty do
3.      while cur ≠ NULL do
4.          push cur;  cur ← cur.left
5.      cur ← pop
6.      append cur.key to out
7.      cur ← cur.right

IS-SORTED(out): main steps
1.  for i ← 0 to out.size − 2 do
2.      if out[i + 1] ≤ out[i] then return FALSE
3.  return TRUE
```

Result: 6,072 codes, strictly increasing, AAA to ZZV, the same minimum and maximum found in Task 2. The full list goes to `data/list_inorder.txt`; the screen shows both ends.

(b) save and load in preorder

Preorder rebuilds the identical tree. The root is written first, so on reload it is inserted first and lands at the root again, and the same holds at every subtree. Inorder would not: it is sorted, and sorted inserts give a chain.

```
PREORDER(out): main steps
1.  if root = NULL then return
2.  stack ← empty;  push root
3.  while stack is not empty do
4.      node ← pop
5.      append node.key to out
6.      if node.right ≠ NULL then push node.right
7.      if node.left  ≠ NULL then push node.left
```

```
SAVE(fileName): main steps
1.  PREORDER(out)
2.  open fileName for writing
3.  for i ← 0 to out.size − 1 do
4.      write out[i] and a newline

LOAD-PREORDER(fileName, records): main steps
1.  clear the tree
2.  open fileName for reading
3.  while getline(in, code) do
4.      find the record whose iata = code
5.      INSERT(code, record)
```

```
SAME-SHAPE(a, b): main steps
1.  if a.height ≠ b.height then return FALSE
2.  LEVEL-ORDER(a, la);  LEVEL-ORDER(b, lb)
3.  if la.size ≠ lb.size then return FALSE
4.  for i ← 0 to la.size − 1 do
5.      if la[i] ≠ lb[i] then return FALSE
6.  return TRUE
```

Result: both trees hold 6,072 nodes at height 177 and their level orders match position by position. The comparison covers all 6,072 nodes, not only the three levels printed.

(c) validate

IS-BST passes each node the range it is allowed to hold. That catches a key correct against its parent but wrong against an ancestor. Pseudocode is in Task 3.

Result: OK after the build, the reload, the insert of QQQ, and each of the three deletes. Inorder after the deletes returns 6,070 codes, still strictly increasing.

Output

```
$ g++ -std=c++17 -O2 TASK4.cpp -o TASK4
$ ./TASK4 data/airports.dat
```

```
Loaded 6072 airports. Height 177.

===== Task 4(a): list, in alphabetical code order =====
  codes listed: 6072
  first 10: AAA AAC AAE AAF AAH AAK AAL AAM AAN AAO
  last  10: ZUM ZVA ZVK ZWA ZWL ZXT ZYI ZYL ZZU ZZV
  strictly increasing: YES
  full list written to data/list_inorder.txt

===== Task 4(b): save in preorder, load it back =====
  preorder codes: 6072
  first 10: GKA AEY AAE AAA AAC ACC ABJ AAR AAL AAH
  last  10: ZTR ZUC ZWA ZVK ZZU ZYL ZWL ZYI ZXT ZZV
> save data/preorder.txt
> load data/preorder.txt
  original  nodes 6072   height 177
  reloaded  nodes 6072   height 177
  top three levels, original:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  top three levels, reloaded:
  level 0: GKA
  level 1: AEY MAG
  level 2: AAE EGS HGU POM
  same shape (height and full level order): YES

===== Task 4(c): validate with the (lo, hi) method =====
  after the build         : OK
  reloaded tree           : OK
  after inserting QQQ     : OK
  after removing QQQ      : OK
  after removing BOS      : OK
  after removing the root : OK
  inorder still sorted    : YES   (6070 codes)
```

Task 5: Range Queries

range FROM TO

```
RANGE(node, from, to, out, visited): main steps
1.  if node = NULL then return
2.  visited ← visited + 1
3.  if node.key > from then RANGE(node.left, from, to, out, visited)
4.  if from ≤ node.key and node.key ≤ to then append node.data to out
5.  if node.key < to then RANGE(node.right, from, to, out, visited)
```

Steps 3 and 5 are the pruning. If node.key $\leq$ from, every key in the left subtree is smaller than node.key, and smaller than from, and the subtree is never entered. If node.key $\geq$ to, the right subtree is skipped for the same reason on the other side. Without those two tests the query would walk all 6,072 nodes.

Left, node, right: the matches come out sorted. `visited` counts every node the query looks at.

Result: `range BOS BWI` returns 113 airports having visited 127 nodes, the two figures in the handout. BOS is first and BWI last, so the range is inclusive at both ends.

Query	Airports	Nodes visited
range BOS BWI	113	127
range AAA AAZ	18	24
range JFK JFK	1	16
range ZZU ZZZ	2	37
range QQA QQZ	1	21

127 nodes out of 6,072: the pruning skipped 5,945. `range JFK JFK` costs 16 nodes, the same as a search for JFK.

The first ten and last ten airports are printed and the complete list of 113 is written to `data/range_BOS_BWI.txt`.

Output

```
$ g++ -std=c++17 -O2 TASK5.cpp -o TASK5
$ ./TASK5 data/airports.dat
```

```
Loaded 6072 airports. Height 177.

===== Task 5: range FROM TO =====

The handout's query. Expected: 113 airports.
> range BOS BWI
  113 airports, 127 nodes visited
    first: BOS   last: BWI

Other ranges, to show the pruning:
> range AAA AAZ
  18 airports, 24 nodes visited
    first: AAA   last: AAZ

> range JFK JFK
  1 airport, 16 nodes visited
    JFK  John F Kennedy International Airport, New York, United States

> range ZZU ZZZ
  2 airports, 37 nodes visited
    ZZU  Mzuzu Airport, Mzuzu, Malawi
    ZZV  Zanesville Municipal Airport, Zanesville, United States

> range QQA QQZ
  1 airport, 21 nodes visited
    first: QQT   last: QQT

range BOS BWI comes out sorted: YES
nodes visited 127 against 6072 in the tree, so the pruning skipped 5945 nodes.

===== range BOS BWI, the airports =====
  BOS  General Edward Lawrence Logan International Airport, Boston, United States
  BOU  Bourges Airport, Bourges, France
  BOW  Bartow Municipal Airport, Bartow, United States
  BOX  Borroloola Airport, Borroloola, Australia
  BOY  Bobo Dioulasso Airport, Bobo-dioulasso, Burkina Faso
  BPC  Bamenda Airport, Bamenda, Cameroon
  BPE  Qinhuangdao Beidaihe Airport, Bagan, Burma
  BPF  Batuna Aerodrome, Batuna, Solomon Islands
  BPG  Barra do Garças Airport, Barra Do Garcas, Brazil
  BPH  Bislig Airport, , Philippines
  ... 93 more ...
  BVI  Birdsville Airport, Birdsville, Australia
  BVS  Breves Airport, Breves, Brazil
  BVY  Beverly Municipal Airport, Beverly, United States
  BWA  Gautam Buddha Airport, Bhairawa, Nepal
  BWB  Barrow Island Airport, Barrow Island, Australia
  BWE  Braunschweig-Wolfsburg Airport, Braunschweig, Germany
  BWF  Barrow Walney Island Airport, Barrow Island, United Kingdom
  BWG  Bowling Green Warren County Regional Airport, Bowling Green, United States
  BWH  Butterworth Airport, Butterworth, Malaysia
  BWI  Baltimore/Washington International Thurgood Marshall Airport, Baltimore, United States
  full list written to data/range_BOS_BWI.txt
```

Task 6: The Shape Experiment

The balanced build and the rebalance command

The sorted array comes from an inorder walk; no separate sort is needed. The middle element as root splits the rest into two equal halves, and repeating that at every level gives a height of $\lfloor \log_2 n \rfloor$.

```
BUILD-BALANCED(a, lo, hi): main steps
1.  if lo > hi then return NULL
2.  mid ← floor((lo + hi) / 2)
3.  node ← new node with key a[mid] and record a[mid]
4.  node.left  ← BUILD-BALANCED(a, lo, mid − 1)
5.  node.right ← BUILD-BALANCED(a, mid + 1, hi)
6.  return node

REBALANCE(): main steps
1.  INORDER-RECORDS(sorted)
2.  BUILD-BALANCED(sorted, 0, sorted.size − 1)
```

Initial call: root ← BUILD-BALANCED(a, 0, n − 1). It halves the array each call, so the recursion is only 12 deep.

```
SHUFFLE(a): main steps
1.  for i ← a.size − 1 down to 1 do
2.      j ← random integer in 0 .. i
3.      swap a[i] and a[j]
```

The table

The same 6,072 codes, four insertion orders, and the same 10,000 random codes searched in every tree.

Build order	Height	Avg depth	10,000 searches	Your explanation
File order	177	18.3	28.0 ms	The file is grouped by country, so long stretches of codes arrive already sorted and build chains inside the tree.
Sorted order	6071	3035.5	3725.9 ms	Every key is larger than the one before, so every insert goes right and the tree is a linked list, $h = n - 1$.
Balanced (middle rule)	12	10.7	21.0 ms	The middle key halves the remaining keys at every level, giving $\lfloor \log_2 6072 \rfloor = 12$.
Random shuffle (bonus)	27	14.0	25.1 ms	A shuffle leaves no long sorted run, so no chain forms and the height stays near $2 \log_2 n$.

Why file order is so much worse than a shuffle

The file lists airports by country, and a country's codes share a prefix. The program counts 364 codes beginning with Y, 324 of them in Canada, and inside that block they arrive in increasing order.

```
LONGEST-INCREASING-RUN(a, startAt): main steps
1.  best ← 1;  cur ← 1;  curStart ← 0;  startAt ← 0
2.  for i ← 0 to a.size − 2 do
3.      if a[i + 1].iata > a[i].iata then
4.          cur ← cur + 1
5.          if cur > best then best ← cur;  startAt ← curStart
6.      else
7.          cur ← 1;  curStart ← i + 1
8.  return best
```

Run on the file it returns 162, from YDA to ZFM, every one of them Canadian. Run on the shuffled array it returns 8.

Inside the tree the first 155 of those codes, YDA to YZR, each go to the right of the one before. They form a single chain running from depth 23 down to depth 177, which is the whole bottom of the tree. The last 7, YZT to ZFM, land under YZS, which was inserted earlier, and start a short branch at depth 23 instead.

A shuffle has no long runs, so no chain forms and the height stays near 27.

What the timings show

Sorted order takes 3,726 ms, balanced takes 21.0 ms. Same keys, same queries, different shape. Average depth is 3035.5 against 10.7.

The other three land within 7 ms of each other because their average depths are all small. A height of 177 costs only the rare query that falls into the Canadian chain.

Output

```
$ g++ -std=c++17 -O2 TASK6.cpp -o TASK6
$ ./TASK6 data/airports.dat
```

```
===== Task 6: the shape experiment =====

  Build order              Height   Average depth  10,000 searches
  ------------------------------------------------------------------
  File order                  177            18.3          28.0 ms
  Sorted order               6071          3035.5        3725.9 ms
  Balanced (middle rule)       12            10.7          21.0 ms
  Random shuffle (bonus)       27            14.0          25.1 ms

> load data/airports.dat
Height: 177   Average depth: 18.3
> rebalance
Height: 12   Average depth: 10.7

  longest already-increasing run, file order : 162  (YDA to ZFM, Canada)
  longest already-increasing run, shuffled   : 8
  codes beginning with Y                     : 364, of which 324 in Canada
```

The last four lines reproduce the handout's sample session: `load` reports height 177 and average depth 18.3, and `rebalance` reports height 12 and average depth 10.7.

Data: OpenFlights.org, `airports.dat`, Open Database License.